Лабораторная работа №6. Квадратурно-амплитудная манипуляция (16QAM, 64QAM, 256QAM)

GNU Radio 3.10 (radioconda). Все схемы проверены компилятором `grcc`, симуляционные ещё и запущены. Численные результаты получены скриптом `tools/sim_lab6.py` (те же блоки без графики), схемы генерируются скриптом `tools/make_lab6.py`.

Пункт	Файл
1.1 – 1.8.4	`lab6_1_16qam.grc`
1.8.5 – 1.8.6	`lab6_1_8_5_16qam_one_symbol.grc`
1.8.7	`lab6_1_8_7_16qam_several.grc`
1.8.8	`lab6_1_8_8_16qam_sdr.grc`
2.1 – 2.1.2	`lab6_2_1_64qam.grc`
2.1.3	`lab6_2_1_3_16qam.grc`
2.1.4	`lab6_2_1_4_64qam_several.grc`
2.1.5	`lab6_2_1_5_64qam_sdr.grc`
3.1	`lab6_3_1_256qam.grc`
3.2	`lab6_3_2_256qam_sdr.grc`

SDR-схемы собраны под стенд из прошлых работ: передатчик HackRF, приёмник RTL-SDR, 433,92 МГц. Без оборудования они проверены только компилятором.

Отличия от рисунков лабника (все остальные блоки и параметры — как на рисунках):

- Блоков Import (`numpy`, `pi`) нет: таблица точек записана без numpy, а величины с π посчитаны числами (см. п. 1.7).

- Добавлен **Throttle** — без SDR иначе процессор загружается на 100 %.
- На передаче в Chunks to Symbols стоит `QAM16.points()` (`QAM64.points()`, `QAM256.points()`) — те же нормированные точки, что ждёт приёмник, вместо «сырой» решётки. Почему — в п. 1.2.
- В 256QAM перед Decimating FIR стоит **Delay = 3** (см. п. 3.1).

Часть 1. 16QAM

1.1 – 1.4. Передатчик (рис. 1–3)

Переменные: `samp_rate = 2M`, `M = 16`, `symb_rate = 50k`.

№	Блок	Параметры
1	Random Source (byte)	Minimum 0, Maximum `M` (выдаёт 0…15), Num Samples 1000, Repeat Yes
2	Chunks to Symbols	таблица `QAM16.points()` — точки объекта созвездия (решётка 4×4, см. ниже)
3	Constellation Sink «Передаваемое»	1024 точки
4	Interpolating FIR Filter	Complex→Complex, Interpolation `int(samp_rate/symb_rate)` = 40, Taps `rrc`
5	RRC Filter Taps `rrc`	Gain 40, Sample Rate `samp_rate`, Symbol Rate `symb_rate`, Excess BW 0,4, Num Taps 40
6	Channel Model	Noise Voltage `Noise`, Frequency Offset `F/samp_rate` (расстройка относительная)
7, 8	QT GUI Range `F`, `Noise`	F: 0…200 кГц, шаг 100 Гц; Noise: 0…5, шаг 0,01
9	Virtual Sink	Stream ID `QAM`

Почему не таблица с рис. 2. Формула `[a+1j*b for a in (numpy.arange(int(M/4))-1.5) for b in ...]` даёт решётку 4×4 с координатами ±0,5 и ±1,5. Объект созвездия на приёме

(рис. 5) нормирует те же точки по амплитуде (средняя амплитуда 1), то есть приёмник ждёт точки в 1,44 раза ближе к центру, чем их передают. Без шума 16QAM всё равно принимается, но у внутренних точек меньше запас до границы решения: в симуляции ошибки появились уже при Noise = 0,5 (0,7 %), а с нормированной таблицей — только с 0,75. Поэтому на передаче взяты `QAM16.points()` — ровно те точки, которые ждёт приёмник. Сама решётка записана в объекте созвездия без numpy: `[a+1j*b for a in [x-1.5 for x in range(4)] for b in [x-1.5 for x in range(4)]]`.

Gain = 40 в передающем фильтре компенсирует интерполяцию: фильтр вставляет 39 нулей между символами, и без усиления амплитуда упала бы в 40 раз.

1.5 – 1.7. Приёмник (рис. 4–5)

№	Блок	Параметры
1	Virtual Source	`QAM`
2	Frequency Sink	FFT 1024, полоса 2 МГц
3	Eye Sink	1024 точки, 40 отсчётов на символ, 2 МГц
4	Decimating FIR Filter	Decimation 40, Taps `rrc_1` — согласованный фильтр + прореживание до 1 отсчёта на символ
5	RRC Filter Taps `rrc_1`	как `rrc`, но **Gain = 1** (прореживание амплитуду не меняет)
6	Constellation Object `QAM16`	Variable Constellation; Symbol Map `list(range(M))`; точки — та же решётка; Rotational Symmetry 4; Dimensionality 1; Normalization Amplitude
7	Constellation Receiver	Constellation `QAM16`, Loop Bandwidth **0.0226**, Min/Max Freq Deviation ∓**0.503**
8	Constellation Sink	выход `symbol`
9	Time Sink	выход `error`
10	Null Sink (3 входа)	выходы `error`, `phase`, `frequency`
11	Number Sink	выход `out` — номер принятого символа

Loop Bandwidth. На вход приёмника приходит 1 отсчёт на символ, значит Fвх = 50 кГц: LB = 4 / (ζ·Туст·Fвх) = 4 / (0,7071 · 0,005 · 50 000) = **0,0226**.

Девиация частоты. GNU Radio задаёт её в радианах на один входной отсчёт: Δω = 2π·Δf / Fвх = 2π · 4000 / 50 000 = **0,503**. Минимальная −0,503, максимальная +0,503. (Значение 0,377 на рис. 4 соответствует 3 кГц: 2π·3000/50 000.)

Блок №10. Схема сначала запускалась без него, как сказано в п. 1.7, и при запуске выдала ошибку `missing connection from output port 2`: выходы phase и frequency обязательно

должны быть куда-то подключены. Поэтому Null Sink добавлен.

1.8. Результаты

1.8.1. Спектр и глазковая диаграмма

- **Главный лепесток спектра — около 100 кГц** (от нуля до нуля ±52 кГц), первый боковой лепесток −26 дБ. Ожидаемая полоса RRC-сигнала Rs·(1+α) = 50 · 1,4 = 70 кГц. Лепесток шире, потому что фильтр из 40 отсчётов длится всего один символ: такой короткий RRC обрезан, и его спектр приближается к спектру прямоугольного импульса длиной T = 1/Rs (нули на ±Rs = ±50 кГц).
- **Гармоники внутри лепестка идут через 50 Гц**. Random Source повторяет одни и те же 1000 символов, поэтому сигнал периодический с периодом 1000/Rs = 20 мс, и спектр состоит из линий с шагом Rs/1000 = 50 Гц (в симуляции на этих частотах 100 % мощности). При FFT 1024 разрешение 2 МГц/1024 ≈ 2 кГц, и линии сливаются. Чтобы их увидеть, нужен FFT не меньше 65 536 (≈ 30 Гц).
- **Глазковая диаграмма:** длительность символа T = 1/50 кГц = **20 мкс** = 40 отсчётов. На экране 2 символа (40 мкс), глаз раскрыт в момент отсчёта через каждые 20 мкс.

1.8.2. Расстояние между символами dE

В решётке соседние точки отстоят на 1 (координаты ±0,5 и ±1,5). После нормировки по амплитуде (средняя амплитуда 1) координаты становятся ±0,33 и ±1,0, и **dE = 0,668**. Тракт передаёт амплитуду с коэффициентом ≈ 0,96, поэтому на созвездии приёмника dE ≈ 0,64. На графике `error` (фазовая ошибка, рад) без шума видна полоса около ±0,18 рад: это межсимвольная интерференция от короткого фильтра на 40 отсчётов.

1.8.3. Частотная расстройка

При плавном увеличении F стрелками синхронизация держится **до 4,0 кГц** и срывается на **4,1 кГц**: оценка частоты приёмника упирается в 4000 Гц и дальше не растёт. **Min/Max Freq Deviation** задают диапазон, в котором петля приёмника может подстраивать частоту. Если расстройка выходит за него, петля не может её скомпенсировать, созвездие начинает вращаться, и символы принимаются неверно. Чем шире диапазон, тем больше допустимая расстройка, но тем легче петле уйти на ложную частоту при шуме. По графику ошибки: до 4 кГц разброс `error` не меняется (СКО ≈ 0,014 рад), после срыва он вырастает до ≈ 0,28 рад — точки «размазываются» по кругу, и dE перестаёт быть различимым.

1.8.4. Шум (расстройка 0)

Noise	0,5	0,75	1,0	1,5	2,0
Ошибки символов	0	0,1 %	1,3 %	12,5 %	73 % (срыв)

Созвездие становится трудно различимым при **Noise ≈ 1,0–1,5**. Шум задаётся на частоте 2 МГц, а согласованный фильтр пропускает только полосу сигнала, поэтому ошибки появляются при довольно больших значениях слайдера.

1.8.5. Один символ (`lab6_1_8_5_16qam_one_symbol.grc`)

Шум = 0. Random Source заменён на **Vector Source** `(0x55,)` → **Packed to Unpacked (4 бита, MSB)**. Байт 0x55 = 0101 0101 разбивается на два одинаковых 4-битных символа **5 и 5**, поэтому на созвездии одна точка. Number Sink показывает **5**, на созвездии одна точка на месте символа 5 — демодуляция верная (в симуляции ошибок нет).

1.8.6. Один символ: расстройка и шум

- Расстройка: ошибки начинаются при **≈ 4,2 кГц** — тот же предел девиации 4 кГц.
- Шум: ошибки появляются при **Noise ≈ 1,0** (0,6 %), при 1,5 — 9 %, при 2,0 происходит срыв.

1.8.7. Несколько значений (`lab6_1_8_7_16qam_several.grc`)

Vector Source `(0x12, 0x34, 0xAB)` → символы **1, 2, 3, 4, 10, 11**. На созвездии 6 точек, приняты ровно те же символы. Расстройка: срыв при ≈ 4,1 кГц; шум: ошибки с Noise ≈ 1,0 (1 %), срыв при 2,0 — так же, как для одного символа.

1.8.8. Весь алфавит через SDR (`lab6_1_8_8_16qam_sdr.grc`)

- Источник: Vector Source `tuple(range(M))` — все 16 символов подряд.
- Передатчик: фильтр → Multiply Const 0,3 → **osmosdr Sink (HackRF)**.
- Приёмник: **osmosdr Source (RTL-SDR)** → **Symbol Sync** (Gardner, PFB с согласованным фильтром) → **AGC** → Constellation Receiver.

Почему схема приёмника отличается: Decimating FIR берёт каждый 40-й отсчёт в фиксированный момент. В симуляции это работает, но у HackRF и RTL-SDR разные тактовые генераторы, и момент отсчёта постоянно уплывает — его подстраивает Symbol Sync. AGC нужен потому, что у QAM информация есть и в амплитуде: уровень принятого сигнала должен совпадать с масштабом точек в объекте созвездия. Если созвездие вращается, расстройка RTL-SDR больше 4 кГц — её нужно убрать слайдером ppm.

Часть 2. 64QAM

2.1. Схема (`lab6_2_1_64qam.grc`)

M = 64, Packed to Unpacked убран, Vector Source `tuple(range(M))`, решётка 8×8: `[a+1j*b for a in [x-3.5 for x in range(8)] for b in ...]`.

Два изменения, без которых 64QAM не принимается:

1. **Формула из лабника** `numpy.arange(int(M/4))-1.5` даёт решётку 4×4 только при M = 16. Для 64 нужна `√M = 8` координат по каждой оси со сдвигом 3,5.
2. **Масштаб передатчика.** Если на передаче оставить «сырую» решётку 8×8, её средняя амплитуда около 3, а приёмник ждёт 1: в симуляции 91 % ошибок даже без шума. С таблицей `QAM64.points()`, как и в 16QAM, ошибок нет.

2.1.1. Шум

Noise	0,2	0,3	0,4	0,5	0,75
Ошибки 64QAM	0	0,3 %	1,9 %	4,7 %	85 % (срыв)

Созвездие становится трудно различимым при **Noise ≈ 0,4–0,5** — примерно в 2–3 раза меньше, чем у 16QAM.

2.1.2. Расстройка

Синхронизация держится **до 4,0 кГц**, срыв на 4,1 кГц — предел задан девиацией приёмника, а не модуляцией. При расстройке близко к пределу точки 64QAM заметно размываются раньше, чем у 16QAM.

2.1.3. Меньшая разрядность (`lab6_2_1_3_16qam.grc`)

Разрядность символа уменьшена с 6 до 4 бит: **M = 16**, решётка 4×4. Шум: ошибки появляются при Noise ≈ 0,75–1,0 вместо 0,3. Расстройка: предел тот же, 4 кГц. **Причина:** при той же средней мощности точки 16QAM стоят в 2 раза дальше друг от друга: dE = 0,668 против 0,329 у 64QAM. Чтобы перепутать соседние точки, шуму нужно сдвинуть точку на dE/2, поэтому 16QAM выдерживает примерно вдвое больший шум (≈ 6 дБ). Зато 64QAM передаёт 6 бит на символ вместо 4 — в 1,5 раза быстрее в той же полосе.

2.1.4. Несколько значений 64QAM (`lab6_2_1_4_64qam_several.grc`)

Vector Source `(5, 27, 42, 63)` — номера символов сразу, без Packed to Unpacked. На созвездии 4 точки, приняты те же значения.

2.1.5. 64QAM через SDR (`lab6_2_1_5_64qam_sdr.grc`)

Так же, как п. 1.8.8, с M = 64. Требования к SDR жёстче: нужны сильный сигнал (запас ОСШ) и точная подстройка ppm.

Часть 3*. 256QAM

3.1. Схема (`lab6_3_1_256qam.grc`)

M = 256, решётка 16×16, Vector Source `tuple(range(M))`, на передаче `QAM256.points()`. Таблица из подсказки `complex(i,q)/170**0.5` нормирует точки по **мощности**, а объект созвездия — по **амплитуде**. Разница около 6 %, а для 256QAM это больше половины расстояния между точками, поэтому взяты те же точки, что в объекте.

Delay = 3 перед Decimating FIR. Канал и фильтры сдвигают центр символа на 3 отсчёта, и без поправки отсчёт берётся немного не в центре. Для 16QAM и 64QAM это даёт ошибку вектора 6 % и не мешает, а для 256QAM (допуск около 3 %) уже приводит к ошибкам. С задержкой 3 отсчёта ошибка вектора 1,6 %, ошибок нет.

Результаты:

- **Шум:** ошибки с Noise ≈ 0,2 (0,3 %), при 0,3 — 6 %, при 0,4 — срыв.
- **Расстройка:** синхронизация срывается уже при **≈ 0,5–1 кГц**, хотя девиация 4 кГц. Петля подстраивает частоту с небольшим запаздыванием по фазе, а у внешних точек 256QAM допуск по фазе всего несколько градусов.
- **Неоднозначность фазы:** созвездие QAM симметрично при повороте на 90°. Иногда приёмник захватывает его повёрнутым: точки на своих местах, но номера символов сдвинуты по кругу. Исправляется дифференциальным кодированием квадрантов.

Сравнение помехоустойчивости:

Модуляция	бит/символ	dE (норм.)	ошибки с Noise ≈	срыв по шуму	срыв по частоте
16QAM	4	0,668	0,75–1,0	2,0	4,1 кГц
64QAM	6	0,329	0,3	0,75	4,1 кГц
256QAM	8	0,164	0,2	0,4	0,5–1 кГц

Каждое увеличение порядка в 4 раза уменьшает dE вдвое: скорость растёт на 2 бита на символ, а устойчивость к шуму и фазовым ошибкам падает.

3.2. 256QAM через SDR (`1ab6_3_2_256qam_sdr.grc`)

Схема как в п. 1.8.8 с M = 256. Для 256QAM нужен очень высокий ОСШ (больше ~30 дБ) и стабильная частота; с RTL-

SDR (8-битный АЦП) чистое созвездие 256QAM, скорее всего, получить не удастся — это ожидаемое ограничение оборудования.

Контрольные вопросы

1. **Структура.** Передатчик: источник данных → Chunks to Symbols (номер символа → точка созвездия) → интерполирующий RRC-фильтр (формирование импульса) → ЦАП/SDR. Приёмник: SDR/АЦП → согласованный RRC-фильтр → символьная синхронизация (прореживание до 1 отсчёта на символ) → Constellation Receiver (подстройка частоты и фазы + решение о символе) → данные.
2. **Constellation Receiver:** Constellation Object — какие точки ожидать и как их нумеровать; Loop Bandwidth — скорость петли фазовой автоподстройки (шире — быстрее захват, но больше шум); Min/Max Freq Deviation — пределы, в которых петля может подстраивать частоту (рад/отсчёт). Выходы: `out` — номер символа, `error` — фазовая ошибка, `phase`, `frequency` — состояние петли, `symbol` — скорректированная точка.
3. **sps и порядок модуляции.** Чем больше sps, тем точнее можно выбрать момент отсчёта и тем лучше работает согласованный фильтр. При высоком порядке (64, 256QAM) точки стоят близко, и даже небольшой сдвиг момента отсчёта (межсимвольная интерференция) приводит к ошибкам, поэтому высоким порядкам нужен большой sps и точная символьная синхронизация.